PROJECT BRIEF / FULL-STACK ENGINEERING

IsdaLog

From the landing port to a verified delivery.

James Ian Bayonas | Sole developer | Independent project

PROBLEM & USERS

Catch registration, price discovery, and cargo handoffs can be difficult to follow at local landing ports. IsdaLog connects those steps in a shared workflow for harvesters, buyers, and riders, with an administrative view for monitoring.

Built for: Harvesters · buyers · logistics riders · administrators

MY CONTRIBUTION

I initiated and built this system independently in response to the lack of transparency I observed in my area. The project brings together the Telegram gateway, marketplace, delivery workflow, and settlement logic.

MAIN TECHNOLOGIES

Laravel 11 / React 18 / Inertia.js / Laravel Reverb / Node.js / Gemini / Ollama / MySQL / SQLite / Redis / Leaflet / Tailwind CSS

VERSION SCOPE

Documented workflow and reviewed settlement code. The walkthrough illustrates the documented 97/3 catch-price split. The linked order controller credits the harvester and rider; it does not explicitly credit the platform share.

WORKFLOW

01. Register the catch

02. Publish and bid

03. Hold funds and dispatch

04. Verify custody

05. Inspect and settle

SYSTEM STRUCTURE

Capture: Node.js Telegram gateway

Classify: Gemini > Ollama fallback

Coordinate: Laravel / Inertia / Reverb

Persist: MySQL or SQLite + Redis

ENGINEERING DECISION

Attempt Gemini inference with a timeout, then fall back to a reachable Ollama instance running llama3.2-vision. Keep wallet changes inside database transactions with row locks.

TRADE-OFF

Local vision inference needs additional compute and a reachable gateway. The fallback reduces dependence on cloud inference, while Telegram and marketplace operations still need connectivity.

READ CASE STUDY | TRY WALKTHROUGH | VIEW SOURCE

Portfolio case studies and walkthroughs provide further detail and sample interactions.